

Building a Knowledge Graph: A Comprehensive End-to-End Guide Using Modern Tools



Brian Curry

Follow

8 min read · 1 day ago



15



Transform unstructured data into intelligent, queryable knowledge structures using today's most powerful open-source and enterprise solutions

...



The explosion of large language models (LLMs) and retrieval-augmented generation (RAG) has thrust knowledge graphs back into the spotlight. What was once a specialized technology used mainly in academic research has become essential infrastructure for enterprises building AI-powered applications. Knowledge graphs now power everything from fraud detection systems to recommendation engines, and they're the secret weapon behind the emerging GraphRAG paradigm that's revolutionizing how we build context-aware AI systems.

This guide walks you through the complete end-to-end process of building a knowledge graph — from raw data to queryable intelligence — using the best tools available today.

What is a Knowledge Graph?

A knowledge graph represents a collection of interlinked descriptions of entities (objects, events, concepts) where each entity is connected by edges that describe the relationships between them. Unlike traditional databases that store data in rigid tables, knowledge graphs model information the way we naturally think about it: as interconnected concepts with meaningful relationships.

For example, instead of storing “Apple” as just a company name in a row, a knowledge graph captures that Apple is a **company**, **headquartered in** Cupertino, **founded by** Steve Jobs, **manufactures** the iPhone, and **competes with** Samsung — all as traversable relationships.

Why Knowledge Graphs Matter for AI

The rise of GraphRAG demonstrates why knowledge graphs have become essential for modern AI applications:

- **Improved Accuracy:** Knowledge graphs provide structured, contextually relevant information that helps RAG systems generate more accurate responses
- **Enhanced Reasoning:** The relational structure enables AI to traverse connections and make logical inferences

- **Reduced Hallucination:** Grounding language models in structured knowledge bases significantly reduces false or inconsistent outputs
- **Explainability:** Explicit relationships make it easier to trace how an AI system arrived at a particular answer
- **Scalability:** Knowledge graphs can be continuously extended with new information over time

. . .

The End-to-End Knowledge Graph Pipeline

Building a knowledge graph involves six major phases:

1. Data Acquisition & Preparation
2. Entity Extraction
3. Relationship Extraction
4. Entity Resolution & Linking
5. Graph Storage
6. Querying & Application Integration

Let's explore each phase with the best tools available.

. . .

Phase 1: Data Acquisition & Preparation

Phase 1: Data Acquisition & Preparation

Understanding Your Data Sources

Knowledge graphs can be built from diverse sources:

- **Unstructured text:** Documents, articles, reports, emails
- **Semi-structured data:** JSON, XML, HTML pages
- **Structured data:** Databases, spreadsheets, APIs
- **Multimedia:** YouTube transcripts, images (via OCR), audio (via speech-to-text)

Text Chunking Strategy

Before extraction, large documents must be split into manageable chunks. The chunking strategy significantly impacts extraction quality.

Recommended approaches:

```
# Using LangChain's sentence splitter
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1024,
    chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " ", ""]
)
chunks = splitter.split_documents(documents)
```

Key considerations:

- **Chunk size:** 512–1024 tokens works well for most LLM extractors
- **Overlap:** 10–20% overlap preserves context across boundaries
- **Semantic coherence:** Try to keep related sentences together

. . .

Phase 2: Entity Extraction

Entity extraction (Named Entity Recognition) identifies the “nodes” of your knowledge graph — the people, places, organizations, concepts, and domain-specific entities in your text.

Open-Source Tools

spaCy (Traditional NLP)

spaCy remains the workhorse for fast, reliable entity extraction with pre-trained models.

```
import spacy
```

```
nlp = spacy.load("en_core_web_lg")  
doc = nlp("Apple Inc. was founded by Steve Jobs in Cupertino,  
California.")
```

```
for ent in doc.ents:
    print(f"{ent.text} -> {ent.label_}")
# Apple Inc. -> ORG
# Steve Jobs -> PERSON
# Cupertino -> GPE
# California -> GPE
```

Pros: Fast, low cost, works offline, deterministic results

Cons: Limited to predefined entity types, struggles with domain-specific entities

GliNER (Zero-Shot NER)

GliNER is a compact transformer model that can identify *any* entity type without fine-tuning — a game-changer for domain-specific knowledge graphs.

```
from gliner import GLiNER
```

```
model = GLiNER.from_pretrained("urchade/gliner_base")
text = "The CRISPR-Cas9 gene editing technique was developed by Jennifer Doudna."
labels = ["technology", "scientist", "research_method"]
entities = model.predict_entities(text, labels)
```

Pros: Flexible entity types, no training required, handles specialized domains

Cons: Slower than spaCy, requires GPU for optimal performance

LLM-Based Extraction (LangChain / LlamaIndex)

For maximum flexibility, use LLMs directly for entity extraction. This approach excels at understanding context and extracting nuanced entities.

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
```

```
extraction_prompt = """
Extract all entities from the following text. For each entity,
provide:
- entity_name: The name of the entity
- entity_type: The type (PERSON, ORG, PRODUCT, CONCEPT, etc.)
- description: A brief description based on context

Text: {text}

Return as JSON array.
"""

llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

Pros: Handles any entity type, understands context deeply, can



Medium



Search



Write



Hybrid Approach (Recommended)

The most effective pipelines combine multiple approaches:

1. Use **spaCy** for standard entities (PERSON, ORG, GPE, DATE)
2. Use **GliNER** or LLMs for domain-specific entities

3. Use LLMs for entity description generation

. . .

Phase 3: Relationship Extraction

Relationships form the “edges” of your knowledge graph — the connections that give meaning to your entities.

The REBEL Model

REBEL (Relation Extraction By End-to-end Language generation) is a state-of-the-art open-source model that extracts both entities and relationships simultaneously.

```
from transformers import pipeline
```

```
triplet_extractor = pipeline(
    'text2text-generation',
    model='Babelscape/rebel-large',
    tokenizer='Babelscape/rebel-large'
)

text = "Steve Jobs co-founded Apple with Steve Wozniak in 1976."
extracted = triplet_extractor(text, return_tensors=True,
max_length=256)
# Extracts: (Steve Jobs, co-founded, Apple), (Steve Wozniak,
co-founded, Apple)
```

LLM-Based Relationship Extraction

For more control over relationship types, LLMs provide unmatched flexibility:

```
KG_EXTRACTION_PROMPT = """
Given the text and entities, extract relationships as triples:
(subject_entity, relationship_type, object_entity)
```

```
Rules:
- Use clear, specific relationship names (FOUNDED_BY,
LOCATED_IN, WORKS_FOR)
- Ensure directionality reflects the actual relationship
- Only extract relationships explicitly stated or strongly
implied

Entities: {entities}
Text: {text}

Return as JSON array of triples.
"""
```

The spacy-llm Integration

For production pipelines, spacy-llm combines spaCy's efficiency with LLM power:

```
# config.cfg for spacy-llm
[nlp]
pipeline = ["llm"]
```

```
[components.llm.task]
@llm_tasks = "spacy.REL.v1"
labels = ["FOUNDED_BY", "WORKS_FOR", "LOCATED_IN",
```

```
"MANUFACTURES"]
```

. . .

Phase 4: Entity Resolution & Linking

Real-world text refers to the same entity in many ways: “Apple,” “Apple Inc.,” “the Cupertino giant,” “the iPhone maker.” Entity resolution merges these into single nodes.

Coreference Resolution

Handle pronouns and references within documents:

```
import spacy
import coreferee
```

```
nlp = spacy.load('en_core_web_lg')
nlp.add_pipe('coreferee')

doc = nlp("Steve Jobs founded Apple. He served as CEO until 2011.")
# Resolves "He" -> "Steve Jobs"
```

Entity Linking to Knowledge Bases

Link extracted entities to canonical identifiers in existing knowledge bases (Wikidata, DBpedia):

```
from relik import Relik
```

```
relik = Relik.from_pretrained("sapienzanlp/relik-entity-linking-base")
result = relik("Apple was founded in California")
# Links "Apple" -> Q312 (Apple Inc. in Wikidata)
# Links "California" -> Q99 (California in Wikidata)
```

LLM-Based Entity Clustering

For custom knowledge graphs, use LLMs to cluster similar entities:

```
# From KGGen approach
clustering_prompt = """
Given these entity names, identify which ones refer to the same real-world
{entities}
```

```
Group synonyms, abbreviations, and variations together.
Return clusters as JSON arrays.
"""
```

. . .

Phase 5: Graph Storage

Open-Source Graph Databases

Neo4j (Most Popular)

Neo4j is the industry standard with the richest ecosystem for knowledge graphs.

```
from neo4j import GraphDatabase
```

```
driver = GraphDatabase.driver("bolt://localhost:7687", auth=
("neo4j", "password"))

with driver.session() as session:
    session.run("""
        MERGE (p:Person {name: $name})
        MERGE (c:Company {name: $company})
        MERGE (p)-[:FOUNDED]->(c)
    """, name="Steve Jobs", company="Apple")
```

Key features:

- Cypher query language
- APOC library for advanced operations
- Built-in visualization (Neo4j Bloom)
- Vector index support for hybrid search
- Free tier available (AuraDB)

NebulaGraph

High-performance distributed graph database, open-source under Apache 2.0:

```
-- nGQL query language
INSERT VERTEX Person(name) VALUES "steve_jobs":("Steve Jobs");
INSERT EDGE founded() VALUES "steve_jobs"->"apple":();
```

Best for: Large-scale deployments, high-throughput requirements

FalkorDB

Ultra-fast graph database optimized for GraphRAG workloads:

```
from falkordb import FalkorDB
```

```
db = FalkorDB()
graph = db.select_graph('knowledge')
graph.query("CREATE (:Person {name: 'Steve Jobs'})-[:FOUNDED]->(:Company {name: 'Apple'})")
```

Best for: Low-latency GraphRAG applications, real-time queries

Memgraph

In-memory graph database with Cypher compatibility:

```
from gqlalchemy import Memgraph
```

```
memgraph = Memgraph()
memgraph.execute("CREATE (:Person {name: 'Steve Jobs'})")
```

Best for: Real-time analytics, streaming data

Enterprise/Cloud Options

Database Provider Best For **Amazon Neptune** AWS AWS-native applications, managed infrastructure **TigerGraph** TigerGraph Large-scale analytics, deep link analysis **Azure Cosmos DB** Microsoft Multi-model, global distribution **Stardog** Stardog Enterprise knowledge graphs, SPARQL

. . .

Phase 6: Building the Complete Pipeline

Option A: Neo4j LLM Knowledge Graph Builder (Easiest)

Neo4j's open-source tool provides a complete no-code solution:

```
# Clone and run
git clone https://github.com/neo4j-labs/llm-graph-builder
cd llm-graph-builder
docker-compose up
```

Features:

- Supports PDFs, web pages, YouTube videos

- Multiple LLM backends (OpenAI, Gemini, Claude, Llama)
- Built-in GraphRAG chatbot
- Schema customization
- Community detection (Leiden algorithm)

Supported LLMs: OpenAI GPT-4/4o, Google Gemini, Anthropic Claude, Meta Llama 3, Diffbot

Option B: LangChain + Neo4j (Flexible)

```
from langchain_experimental.graph_transformers import LLMGraphTransformer
from langchain_openai import ChatOpenAI
from langchain_community.graphs import Neo4jGraph
```

```
# Initialize
llm = ChatOpenAI(model="gpt-4o", temperature=0)
transformer = LLMGraphTransformer(llm=llm)
graph = Neo4jGraph(url="bolt://localhost:7687",
username="neo4j", password="password")

# Transform documents to graph
documents = load_documents("data/")
graph_documents =
transformer.convert_to_graph_documents(documents)

# Store in Neo4j
graph.add_graph_documents(graph_documents)
```

Option C: LlamaIndex PropertyGraph (Full Control)

```
from llama_index.core import PropertyGraphIndex
from llama_index.llms.openai import OpenAI
from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore
```

```
# Setup
graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="password",
    url="bolt://localhost:7687"
)

# Build index with extraction
index = PropertyGraphIndex.from_documents(
    documents,
    llm=OpenAI(model="gpt-4o"),
    property_graph_store=graph_store,
    show_progress=True
)
```

Option D: Microsoft GraphRAG (Research-Grade)

Microsoft's GraphRAG library implements the full pipeline from their research paper:

```
pip install graphrag
graphrag init --root ./ragtest
graphrag index --root ./ragtest
graphrag query --root ./ragtest --method global "What are the main themes"
```

Unique features:

- Hierarchical community detection (Leiden algorithm)

- Community summarization for global queries
- Local + global retrieval strategies

• • •

Implementing GraphRAG

Once your knowledge graph is built, you can implement sophisticated retrieval:

Local Retrieval (Entity-Focused)

```
from neo4j_graphrag.retrievers import VectorCypherRetriever
```

```
retriever = VectorCypherRetriever(
    driver=driver,
    index_name="entity_embeddings",
    retrieval_query="""
        MATCH (node)-[r]-(neighbor)
        RETURN node.name, type(r), neighbor.name
        LIMIT 10
    """
)
```

Global Retrieval (Community Summaries)

```
# Query community summaries for thematic questions
community_query = """
```

```
MATCH (c:Community)
WHERE c.summary CONTAINS $keyword
RETURN c.summary, c.entities
ORDER BY c.relevance DESC
""""
```

Hybrid Approach

```
from neo4j_graphrag.generation import GraphRAG
from neo4j_graphrag.retrievers import HybridRetriever
```

```
# Combine vector + graph traversal + full-text
retriever = HybridRetriever(
    driver=driver,
    vector_index="chunks",
    fulltext_index="content",
    retrieval_query="MATCH (n)-[:RELATED_TO*1..2]-(m) RETURN
m"
)

rag = GraphRAG(retriever=retriever, llm=llm)
response = rag.search("What products did Apple release in
2023?")
```

• • •

Best Practices & Recommendations

Schema Design

1. **Start simple:** Begin with core entity types and relationships
2. **Use domain experts:** Involve subject matter experts in schema definition
3. **Iterate:** Refine your schema based on extraction results
4. **Document everything:** Maintain clear definitions for entity types and relationships

Cost Optimization

Document Count Recommended Approach < 100 LLM extraction
with GPT-4o 100–1,500 LLM extraction with prompt engineering >
1,500 Fine-tune smaller models or use hybrid approach

Quality Assurance

1. **Sample validation:** Manually review 5–10% of extractions
2. **Entity resolution:** Invest time in deduplication
3. **Feedback loops:** Build mechanisms to correct errors
4. **Continuous updates:** Knowledge graphs require ongoing curation

. . .

Tool Comparison Matrix

Tool	Type	Best For	Learning Curve	Cost
Neo4j LLM Graph Builder	End-to-end	Quick start, prototyping	Low	Free (AuraDB free tier)
LangChain + Neo4j Framework	Custom pipelines	Medium	LLM API costs	
LlamaIndex Framework	RAG applications	Medium	LLM API costs	
Microsoft GraphRAG Library	Research, global queries	High	LLM API costs	
spaCy + REBEL NLP	Cost-effective, offline	Medium	Free	
Diffbot	Commercial Web-scale extraction	Low	Enterprise pricing	

. . .

Conclusion

Building a knowledge graph has never been more accessible. The combination of powerful LLMs for extraction, mature graph databases for storage, and frameworks like LangChain and LlamaIndex for orchestration means you can go from raw documents to a queryable knowledge graph in hours, not months.

Start here:

1. **Quickest path:** Use Neo4j's LLM Knowledge Graph Builder with the free AuraDB tier
2. **Most flexible:** Combine LangChain's LLMGraphTransformer with Neo4j
3. **Production-ready:** Build custom pipelines with LlamaIndex PropertyGraph
4. **Research-grade:** Implement Microsoft's GraphRAG for complex analytical queries

The future of AI lies in grounding language models with structured knowledge. Knowledge graphs provide that grounding — making AI more accurate, explainable, and trustworthy. The tools are ready. Your data is waiting.

. . .

What knowledge graph will you build? Share your projects and questions in the comments below.

. . .

About the Author: Brian Curry is the creator of PyCausalSim, an AI and ML Research Scientist, and founder of Vector1 AI, helping companies move from correlation-based analytics to causal decision-making.

Resources:

- [Neo4j LLM Graph Builder](#)
- [LangChain Graph Transformers](#)
- [LlamaIndex Knowledge Graph Guide](#)
- [Microsoft GraphRAG](#)
- [Awesome-GraphRAG](#)

Ontology

Knowledge Graph

Artificial Intelligence

LLM

Brian Curry



Written by Brian Curry

392 followers · 17 following

Follow



AI Strategist and Developer & Marketing Economist. Founder, Vector1. Building causal systems, cognitive AI architecture, & economic models for large orgs

More from Brian Curry

 Brian Curry

The Journey to Causality: From Dashboards to Causal Inference

Introduction

Dec 16, 2025  119  2  

 Brian Curry

Entity Graphs: How to Develop, Analyze, and Visualize...

Introduction: Why Entity Graphs Matter

Oct 3, 2025  256  8  

 Brian Curry

A Practical Guide to Causal Inference in Marketing Chann...

Why Most Marketing Analytics Gets It Wrong (And How to Fix It)

Nov 20, 2025  40  

 Brian Curry

Mapping the Hidden Architecture of Your...

How machine learning can transform the invisible patterns in your company's...

Dec 26, 2025  3  

See all from Brian Curry

Recommended from Medium



Fanghua (Joshua) Yu

A Unified Framework for AI-Native Knowledge Graphs

Part 1—Generative Knowledge Modeling

4d ago



22



1



Sriram Narasimhan

Building Blocks of Semantics: Ontologies, Knowledge Graph...

The Semantic Layer: The Trust Fabric of the Agentic Enterprise (Part 2)

Jan 6



17





In GoPenAI by Induwara Gayashan

Giving Memory to Knowledge: Building Persistent Knowledg...

A Hands-On Guide to Persisting and Exploring Knowledge Graphs with Neo4j.

Jan 8



4



Reliable Data Engineering

Claude Cowork: Your AI Coworker Just Showed Up For...

3d ago



36



1



Ruchi Toshniwal

Demystifying LLMs (3/8): Contextual Embeddings

How Transformers Give Static Word Representations Dynamic Meaning

Sep 4, 2025



Vishal Mysore

Agentic Knowledge Graphs: Visualizing AI Reasoning in...

An Agentic Knowledge Graph represents a fundamental shift in how we visualize...

5d ago



168



1



See more recommendations

